

An Introduction to Programming through C++

Abhiram G. Ranade

Lecture 6.1

Ch. 10.3: Virahanka Numbers

Virahanka Numbers

- Virahanka was an ancient India prosodist (6th-8th century AD).
- Prosodists study patterns of rhythm and sound in poetry.
- Viharanka asked a question about poetic meters and solved it using recursion.
- A poetic meter is characterized by
 - Number of syllables in the meter
 - The duration of each syllable: short(duration 1), or long (duration 2)
 - Example: SLSSLS is a poetic meter with 6 syllables, total duration 8.

Example of a poetic meter

- “Shardulvikridit”
- Ya kun den du tu shar Haar dhawala yashubh ra vas tra vru ta
- L L L S S L S L S S S L L L S L L s L

19 syllables, Duration 30.

Virahanka's question

“How many poetic meters exist of total duration D ?”

- $D = 1$: $\{S\}$
- $D = 2$: $\{SS, L\}$
- $D = 3$: $\{SSS, SL, LS\}$
- $D = 4$: $\{SSSS, SSL, SLS, LSS, LL\}$

Let $V(D)$ denote the number of poetic meters of duration D .

- We have $V(1) = 1$, $V(2) = 2$, $V(3) = 3$, $V(4) = 5$.

Virahanka wondered whether there is an easy way to calculate $V(D)$.

- Next: nice connections to recursion!

Virahanka's Solution

The first syllable of every meter must be S or L.

$S(D)$ = Set of meters of duration D with first syllable S.

$L(D)$ = Set of meters of duration D with first syllable L.

$$V(D) = |S(D)| + |L(D)|$$

Key Question: Suppose I remove the first letter from every meter in $S(D)$, what remains?

- The meters that remain will have duration $D-1$.
- All possible meters of duration $D-1$ and only those will now be present in $S(D)$.
- So $|S(D)| = V(D-1)$

If I remove the first letter from all meters in $L(D)$:

- Each meter that remains will have duration $D-2$.
- All meters of duration $D-2$ will be present.
- So $|L(D)| = V(D-2)$
- **Observation:** $V(D) = V(D-1) + V(D-2)$ for $D > 2$.

What we have discussed

- Virahanka's problem: Find the number $V(D)$ of poetic meters having duration D .
- We can work out by hand: $V(1) = 1$, $V(2) = 2$
- Working out larger $V(D)$ is tedious and error-prone.
- We also know $V(D) = V(D-1) + V(D-2)$ for all $D > 2$.
- Next: A program to calculate $V(D)$



Program to compute $V(D)$

- Natural to use a recursive function.
- For $D > 2$ we should use $V(D) = V(D-1) + V(D-2)$
- Clearly $D=1$, $D=2$ should be base cases.

Does this satisfy our 4 requirements?

```
int V(int D){// Precondition: D > 0
    if(D == 1) return 1;
    else if(D == 2) return 2;
    else return V(D-1) + V(D-2);
}
```

- Is the correct value returned for the base cases?
- Do the level 1 calls satisfy the precondition?
 - Level 1 calls made only if $D > 2$.
- Does the “problem size” reduce? Can it reduce indefinitely?
 - D reduces in each call, but cannot go below 1.
- If the level 1 calls return the correct value, will the top level call return the correct value?
 - $V(D) = V(D-1) + V(D-2)$ for $D > 2$.

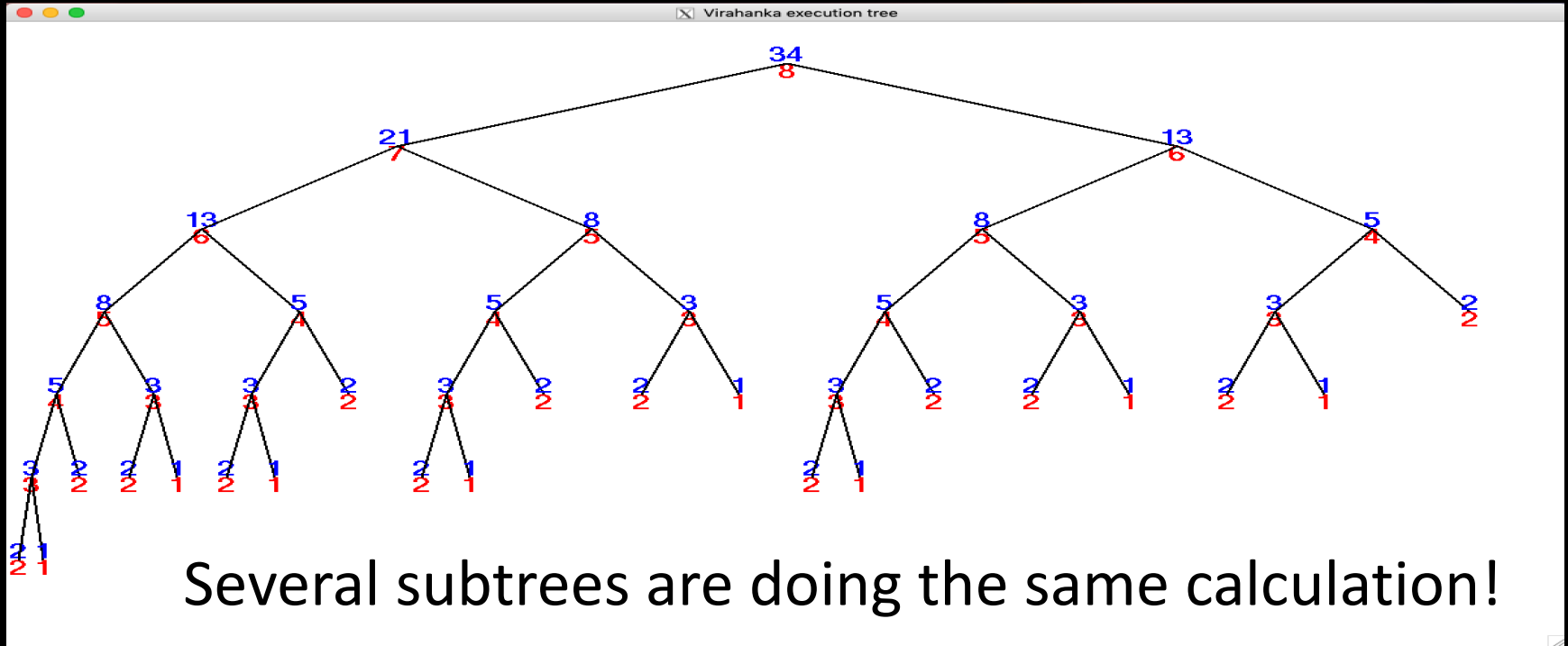
Demo

- Virahanka.cpp
- Observation:
 - Beyond $D=45$, the time for $V(D)$ seems to increase a lot.

Understanding the execution $V(D)$

- The execution of any recursive program can be visualized by drawing its “Execution tree” or its “Recursion tree”.
- The execution tree is like the trees seen earlier.
- The root corresponds to the original call, say $V(10)$.
- $V(10)$ will make recursive calls to $V(9)$ and $V(8)$.
- So we will have branches going out of the root to “nodes” for $V(9)$ and $V(8)$.
- From those we will have further branches according to the further recursive calls that get made.
- No branches leave the nodes corresponding to $V(1)$, $V(2)$.

Observations from Vtree.cpp



Several subtrees are doing the same calculation!

What we have discussed

- Recursive function to calculate Virahanka numbers $V(D)$.
- The function takes a lot of time for $D > 45$.
- Execution tree or recursion tree show that we are performing the same calculation several times.
- Next: How to avoid duplication of work



Can we avoid duplication of work?

- Once we calculate something, do not calculate it again.
- Our function knows $V(1) = 1$, $V(2) = 2$
- It first calculates $V(3) = V(1) + V(2)$
- Later calculates $V(4) = V(3) + V(2)$
- ...
- Use a loop: In iteration i , $i=3..D$ we will calculate $V(i)$.
- For this we will need to remember $V(i-1)$, $V(i-2)$ calculated earlier
- Let us use variables `viminus1`, `viminus2` for this.
- We will use a variable `vi` in which to have $V(i)$.

Non recursive calculation of Virahanka numbers

- For iteration 3, we need
 - $v_{\text{minus}1} = V(2) = 2$,
 - $v_{\text{minus}2} = V(1) = 1$.
- At the beginning of the iteration we calculate
 - $v_i = v_{\text{minus}1} + v_{\text{minus}2}$
- For the next iteration we need
 - $v_{\text{minus}2} = v_{\text{minus}1}$ and
 - $v_{\text{minus}1} = v_i$
- The final result is in v_i
- This works only for $D \geq 2$.

Remarks

- The natural recursive algorithm might be very slow.
- By examining what the algorithm does, we might be able to discover a better algorithm.
- Hint: take ideas from basic Virahanka algorithm and tree drawing algorithm.



Nim Game

- Game starts with 3 stacks of coins, A, B, C
- Say n_A , n_B , n_C are the number of coins in these stacks.
- Player 1 and 2 alternately make a move.
- Valid move: choose one of the stacks and remove any number of coins from it.
- If a player is left with no possible move, they lose.

Nim Game

- Given numbers, n_A , n_B , n_C , can we find out whether it is a winning position?
- (n_A, n_B, n_C) is called a winning position if there is a strategy for the next player to win from this position.
- (n_A, n_B, n_C) is called a losing position if no matter what you play, the opponent has a winning strategy.

Nim Game

- How to decide if a given position (nA, nB, nC) is winning ?
- Answer:
 - Consider all possible positions which can be obtained from (nA, nB, nC) in one move.
 - If any one of them is losing, then (nA, nB, nC) is winning.
 - Otherwise (nA, nB, nC) is losing.
- Base Case: $(0, 0, 0)$ is losing.
- Clearly, the above definition is recursive. Hence, can be computed using recursion.

An Introduction to Programming through C++

Abhiram G. Ranade

Lecture 6.2

Ch. 11: Program Organization and Functions

How do we write large programs?

- Designing/building anything large becomes easier if we can think of it as made of small parts.
- Example:
 - A book is made of chapters.
 - A car is made of many subsystems
- For programs, the smaller parts = functions.
 - We break up the overall requirements into separate, small, somewhat independent computations
 - Code up the separate parts as separate functions.

An aside: the main program is also a function

- In C++ the standard way to write the main program is to write it as a function.
 - The function must be named `main`.
 - Its return type must be `int`.
 - For now, it does not take any arguments.

Instead of `main_program{ xxx }`, you would write: `int main(){ xxx }`
- Simplecpp translates `main_program` into the phrase “`int main()`”
- Simplecpp provides this feature so that you don't need to understand functions etc. on the first day.
- The function `main` is allowed not to have a return statement.
 - The value returned has little consequence anyway.
- From now on we will not use `main_program`, but use `main`.

Why dividing code into functions helps

- Different people can write different functions, so work can be divided.
- Functions are self documenting:
 - Function name, parameters, give clues as to what it does.
 - Even better if you write the specification as a comment..
 - In a long main program it is not obvious where/how to put comments.
- “Write a little bit of code, test it, write some more, ...”
 - Test your program one function at a time
- A function is a good way to package and share code.
 - If you have written some code and want to give it to someone else, give them a file containing a function containing your code.

Physical units of code: files

- Suppose several people write different functions of the same program.
 - Convenient for each to use a different file.
- How will a function in one file call a function in other files?
 - Answers soon.

“But am I going to write large programs in this course?” -

1

- The amount code you write in any assignment in this course will be at most 150 lines or so.
- You should not write 150 lines as a single main program.
- Max size of main program or any function < 20 lines
 - 20 lines you can see at a glance
 - If you can see a function at a glance, you are more likely to understand it fully and write it without errors
 - Several experienced programmers recommend even smaller functions.
- So you should break large code into small functions.
 - Details when we consider problems which require large programs.

What we discussed

- The main program is a function
- Need for splitting a program into many functions
- Need for splitting a program into many files
- What we will see next:
 - How to split programs into files
 - How your program uses code written by others
 - Using C++ without simplecpp



Splitting a program into many files

- A program may contain several functions.
 - All need not be placed in the same file.
- If code in file F calls a function f:
 - function f must be **declared** inside F, textually before any call to it.
- A function definition is a declaration.
 - But there can be other ways to declare. Next.
- Every function must be defined in just one of the files that are used for a program.

Function declaration

- A function declaration is essentially the definition without the body.
- Example: declaration of gcd function: `int gcd(int m, int n);`
- Also acceptable: `int gcd(int, int);`
- The declaration tells the compiler that if the name gcd appears later, it will be a function and take 2 arguments of the specified type.
 - Compiler can now check if the name is used correctly in the rest of the file.
- If a file contains a call to a function but does not contain its definition:
 - It can only be partially compiled into an **object module**.
 - To get an executable program, all the object modules containing all called functions must be **linked** together.

Other names for “declaration” : **Signature, Prototype**

- Example next.

Example of code split over multiple files

- File gcd.cpp

```
int gcd(int m, int n){ ... }
```

- File lcm.cpp

```
int gcd(int, int);  
int lcm(int m, int n){  
    return m*n/gcd(m,n);}
```

- File main.cpp

```
int lcm(int, int);  
int main(){  
    cout << lcm(36,24) << endl;  
}
```

- **Function definitions**, **function declarations**

- Each file has declarations of called functions.
- To compile and link all files together:

```
s++ main.cpp lcm.cpp gcd.cpp
```

- To compile each file separately:

```
s++ -c lcm.cpp
```

- -c : produce lcm.o (object module).

- To get executable from Object modules:

```
s++ main.o lcm.o gcd.o
```

What you typically do: s++ pgm.cpp m1.o m2.o

- Compile your program pgm.cpp
- Link it to object modules developed by others

Your pgm.cpp must have the right declarations...

Header files

- Tedious to remember what declaration to include in each file.
- Instead, we can put all declarations into a header file, and “include” the header file into every file.

- Header file gcdlcm.h

```
int gcd(int, int);  
int lcm(int, int);
```

- The directive “#include filename”
 - gets replaced by the content of the named file.
 - It is acceptable if we declare functions that do not get used.
 - It is acceptable if we have both a declaration and then the definition of a function in the same file.

- File gcd.cpp

```
#include “gcdlcm.h”  
int gcd(int m, int n){ ... }
```

- File lcm.cpp

```
#include “gcdlcm.h”  
int lcm(int m, int n){ ... }
```

- File main.cpp

```
#include <simplecpp>  
#include “gcdlcm.h”  
int main(){ ... }
```

More on header files

- Header files customarily have the suffix .h or .hpp., or no suffix.
- If header file is mentioned in “ ”, it is picked up from the current directory.
- If it is mentioned in < >, it is picked up from some standard place, e.g. `simplecpp`

What we discussed

- How to split a program over many files
- How to assemble a program out of functions in many files
- Function declarations and definitions
- Header files
- Next: Namespaces



Namespaces: High level ideas

- Suppose many people cooperatively develop a single program.
 - Possible that several people may define function with the same name.
- Creates conflict/ambiguity.
- Can be avoided using namespaces.
- Namespace = catalog of names.
- The “full name” of a function f defined in a namespace N is $N :: f$
- Suppose f is defined in two namespaces N and P .
 - We can specify which we mean by writing $N :: f$ or $P :: f$.

Defining a namespace

```
namespace N{  
    declarations/definition of names  
}
```

- This creates a namespace with name N, and also defines/declares names inside it.
- You can add more names to a namespace N simply by writing `namespace N{ }` again.
- A name g defined without putting it inside a namespace is said to belong to the global namespace. Its fullname is `::g`.

Example

```
namespace N{  
    int gcd(int m, int n){ ... }  
    int lcm(int m, int n){ ... }  
}  
int main(){  
    cout << N::lcm(36,24) << endl;  
}
```

The `using` directive

- Suppose you refer to names defined in some namespace `N` very frequently.
 - You may find it tedious to write `N::` all the time.
- Put the following line at the top of your program
`using namespace N;`
- Then you will be allowed to use any name from `N` without having to write `N::` before it.

Example using using

```
namespace N{  
    int gcd(int m, int n){ ... }  
    int lcm(int m, int n){ ... }  
}  
using namespace N;  
int main(){  
    cout << lcm(36,24) << endl;  
}
```

What we discussed

- Namespaces
 - Helps many people use the same name and yet link their work together if needed
- The `using` directive
- Next:
 - How to use C++ without `simplecpp`
 - Concluding remarks on this lecture sequence



Using C++ without simplecpp

- If you use C++ without simplecpp, you will not be able to do graphics.
- Also, when you write `#include <simplecpp>` it itself includes the following lines for you:

```
#include <iostream>
#include <cmath>
using namespace std;
```

- These lines are useful:
 - The names `cin`, `cout`, `endl` are defined in the namespace `std`, in the standard header file `iostream`.

If you do not include `simplecpp`, be sure to write these lines, that is enough!

- The header file `cmath` includes math functions such as `sqrt`, `sin`, `abs`.

Simple example: Using C++ without simplecpp

```
#include <iostream>
using namespace std;
```

```
int main(){
    int n;
    cin >> n;
    cout << n*n*n <<
        endl;
}
```

```
#include <iostream>
```

```
int main(){
    int n;
    std::cin >> n;
    std::cout << n*n*n <<
        std::endl;
}
```

Demo

- withoutSimplecpp.cpp
 - Uses using directive
- withoutSimplecpp2.cpp
 - Does not use the using directive
- These may be compiled using the basic C++ compiler, without loading any of the simplecpp header files and libraries.

Concluding Remarks

- Functions are building blocks of programs.
- Functions can be put into many files, provided each file contains a declaration before the use.
- Declarations go into header files.
- Names can be put inside namespaces
- Different programmers can put their work in different namespaces; same name may be defined in many namespaces.
- Details discussed in the book.

